

Сейчас мы с вами научимся работать с флагами. Флаг - это специальная переменная, которое может принимать только два значения: *true* и *false*. С помощью флагов можно решать задачи, проверяющие отсутствие чего-либо: к примеру, можно проверить, что в массиве нет элемента с определенным значением. Давайте приступим.

Флаги

Давайте решим следующую задачу: дан массив, нужно проверить, есть ли в нем элемент со значением 'с' или нет. Если есть - выведем 'есть', если нет - выведем 'нет'.

Для начала давайте попробуем вывести 'есть'. Для этого переберем все элементы нашего массива и ифом будем спрашивать - равен ли текущий элемент значению 'с'. Если равен - выведем 'есть':

```
var arr = ['a', 'b', 'c', 'd', 'e'];

for (var i = 0; i < arr.length; i++) {
    if (arr[i] == 'c') {
        alert('Есть');
    }
}
```

Но наше решение не очень хорошо: ведь если в массиве будет не одно значение 'с', а несколько, то алерт выведется несколько раз. Давайте исправим наш массив (сделаем два элемента с 'с') и убедимся в этом:

```
var arr = ['a', 'b', 'c', 'd', 'c'];

for (var i = 0; i < arr.length; i++) {
    if (arr[i] == 'c') {
        alert('Есть'); //выведет несколько раз
    }
}
```

Поправим проблему: оборвем цикл с помощью **break**, если элемент уже найден:

```
var arr = ['a', 'b', 'c', 'd', 'c'];
for (var i = 0; i < arr.length; i++) {
    if (arr[i] == 'c') {
        alert('Есть');
        break; //оборвем цикл
    }
}
```

Давайте теперь попробуем сделать так, чтобы, если в массиве вообще нет элементов со значением 'с', выводилось 'нет'. Распространенным заблуждением будет добавить **else** к нашему ифу - в этом случае 'нет' будет выводиться на все элементы, не являющиеся 'с':

```

var arr = ['a', 'b', 'c', 'd', 'c'];
for (var i = 0; i < arr.length; i++) {
    if (arr[i] == 'c') {
        alert('Есть');
    } else {
        alert('Нет'); //выведет на 'a', 'b' и 'd'
    }
}

```

Итак, идея добавить else - плохая идея, не рабочая. Для решения задач подобного типа (такие задачи встречаются достаточно часто) используют так называемые *флаги*.

Флаг - это такая переменная, которая может принимать два значения: true или false. Как работать с флагами - посмотрим на примере нашей задачи.

Итак, давайте сделаем переменную **flag** с таким значением: если она равна true - то в массиве есть элемент 'c', а если false - то такого элемента нет.

Изначально поставим переменную **flag** в значение false - то есть будем считать, что элемента 'c' в массиве нет:

```

var arr = ['a', 'b', 'c', 'd', 'c'];
var flag = false; //считаем, что элемента 'c' нету в массиве

```

Затем запустим цикл с ифом так, как мы делали это раньше. Если цикл обнаружит, что в массиве есть элемент 'c' - то поставим переменную **flag** в значение true (и выйдем из цикла с помощью break).

```

for (var i = 0; i < arr.length; i++) {
    if (arr[i] == 'c') {
        flag = true; //элемент есть - переопределим переменную flag
        break;
    }
}

```

Далее: ответ на вопрос, если в массиве 'c' или нет, мы можем дать только после цикла. И этот ответ у нас уже есть: после цикла переменная **flag** могла остаться false или могла сменить свое значение на true, если цикл обнаружил в массиве 'c':

```

var arr = ['a', 'b', 'c', 'd', 'c'];
var flag = false; //считаем, что элемента 'c' нету в массиве

for (var i = 0; i < arr.length; i++) {
    if (arr[i] == 'c') {
        flag = true; //элемент есть - переопределим переменную flag
        break;
    }
}

```

//тут переменная flag равна или true, или false

Теперь после цикла мы можем сделать иф, который посмотрит на переменную flag и выведет на экран 'есть' или 'нет':

```
var arr = ['a', 'b', 'c', 'd', 'c'];
```

```
var flag = false; //считаем, что элемента 'c' нету в массиве
```

```
for (var i = 0; i < arr.length; i++) {  
    if (arr[i] == 'c') {  
        flag = true; //элемент есть - переопределим переменную flag  
        break;  
    }  
}
```

```
if (flag === true) {  
    alert('Есть');  
} else {  
    alert('Нет');  
}
```

Таким образом и работают флаги. Запомните этот прием - он обязательно встретится вам в дальнейшем, причем неоднократно.

Флаги в функции

Когда мы работаем с пользовательскими функциями - флаги тоже можно и нужно использовать, но работа с ними упрощается. Давайте решим ту же задачу на 'c', но таким образом: необходимо создать функцию **hasElem**, которая параметром будет принимать массив и возвращать true, если 'c' есть в массиве, и false - если нет:

```
var arr = ['a', 'b', 'c', 'd', 'c'];
```

```
alert(hasElem(arr)); //выведет true или false
```

Давайте реализуем функцию **hasElem**. В этом случае переменная flag и команда break нам не понадобятся - вместо них все сделает команда return.

Как это будет: внутри функции запускаем наш цикл с ифом и, если мы попали в иф - делаем **return true**, тем самым выходя из цикла и из функции. А если цикл прокрутился, но выхода из функции не случилось - после цикла делаем **return false**:

```
function hasElem(arr) {  
    for (var i = 0; i < arr.length; i++) {  
        if (arr[i] == 'c') {  
            return true;  
        }  
    }  
}
```

```
        return false; //элемент 'с' не найден
    }
    var arr = ['a', 'b', 'c', 'd', 'c'];
    alert(hasElem(arr));
```

Еще раз как это работает: если в массиве найдется элемент со значением 'с' - мы выйдем из функции с помощью return. Но если в массиве не найдется элемент 'с' - выхода из функции не произойдет и выполнение дойдет до команды return false. И получится, что функция вернет false в знак того, что элемент 'с' не найден в массиве.

Задачи для решения

На флаги

- 1 Дан массив с числами. Проверьте, что в этом массиве есть число **5**. Если есть - выведите 'да', а если нет - выведите 'нет'.
- 2 Дано число, например **31**. Проверьте, что это число не делится ни на одно другое число кроме себя самого и единицы. То есть в нашем случае нужно проверить, что число 31 не делится на все числа от 2 до 30. Если число не делится - выведите 'false', а если делится - выведите 'true'.
- 3 Дан массив с числами. Проверьте, есть ли в нем два одинаковых числа подряд. Если есть - выведите 'да', а если нет - выведите 'нет'.

Приемы работы с логическими значениями

Суть приемов: различные логические операции типа >, <, ==, ===, != могут использоваться не только в ифах, но и сами по себе. Пример:

```
var a = 4;
var b = 3;
```

```
var c = a > b; //в переменную c запишется true
```

Можно также делать комбинации с логическим И && и ИЛИ ||:

```
var a = 3;
var b = 4;
```

```
var c = a < 10 && b < 10;
```

Логические операции без ифов

Пусть у нас есть функция с ифом. Вот она:

```
function func(a, b) {  
    if (a > b) {  
        return true;  
    } else {  
        return false;  
    }  
}
```

Вооружившись знаниями из начала урока, ее можно переписать гораздо проще:

```
function func(a, b) {  
    return a > b;  
}
```

Логическое ИЛИ

Логическое ИЛИ || работает немного не так, как мы ожидаем. Изучите примеры:

```
var c = false || 5; //в c запишется 5  
var c = 5 || false; //в c запишется 5  
var c = 5 || 4; //в c запишется 5  
var c = 4 || 5; //в c запишется 4
```

Вместо false может быть пустая строка, 0, undefined, null, или вообще любое значение, которое в логическом контексте является false.

Таким образом с помощью || можно сделать выбор между двумя значениями. Если одно из них false или приводится к нему - то в переменную будет записано второе.

Задачи

4 Сделайте функцию, которая параметрами принимает 2 числа. Если эти числа равны - пусть функция вернет true, а если не равны - false.

5 Сделайте функцию, которая параметрами принимает 2 числа. Если их сумма больше 10 - пусть функция вернет true, а если нет - false.

6 Сделайте функцию, которая параметром принимает число и проверяет - отрицательное оно или нет. Если отрицательное - пусть функция вернет true, а если нет - false.

Практика на массивах

Заполнение массивов

Давайте заполним массив десятью иксами 'x'. Для этого воспользуемся методом push:

```
var arr = [];  
for (var i = 0; i < 10; i++) {  
    arr.push('x');  
}
```

```
console.log(arr); //выведет ['x', 'x', 'x', 'x', 'x', 'x', 'x', 'x', 'x', 'x']
```

Давайте теперь заполним массив числами от 1 до 10:

```
var arr = [];  
for (var i = 1; i <= 10; i++) {  
    arr.push(i);  
}
```

```
console.log(arr); //выведет [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Можно не использовать **push**, а указать ключи напрямую:

```
var arr = [];  
for (var i = 0; i < 10; i++) {  
    arr[i] = 'x';  
}
```

```
console.log(arr); //выведет ['x', 'x', 'x', 'x', 'x', 'x', 'x', 'x', 'x', 'x']
```

```
var arr = [];  
for (var i = 0; i < 10; i++) {  
    arr[i] = i + 1;  
}
```

```
console.log(arr); //выведет [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Переворот массива

Давайте из массива ['a', 'b', 'c', 'd', 'e'] сделаем ['e', 'd', 'c', 'b', 'a']. Для этого переберем исходный массив с конца и запишем его элементы в новый массив - они будут идти в обратном порядке:

```
var arr = ['a', 'b', 'c', 'd', 'e'];
var result = [];

for (var i = arr.length - 1; i >= 0; i--) {
    result.push(arr[i]);
}

console.log(result); //выведет ['e', 'd', 'c', 'b', 'a']
```

Переворот объекта

Давайте поменяем ключи и значения в объекте, например из {a: 1, b: 2, c: 3, d: 4, e: 5} сделаем {1: 'a', 2: 'b', 3: 'c', 4: 'd', 5: 'e'}.

Для этого переберем циклом **for-in** исходный объект и создадим при этом новый объект **result**. Ключами нового объекта сделаем элементы старого (это obj[key]), а значениями нового объекта - ключи старого (это key):

```
var obj = {a: 1, b: 2, c: 3, d: 4, e: 5};
var result = {};

for (var key in obj) {
    result[obj[key]] = key;
}

console.log(result); //выведет {1: 'a', 2: 'b', 3: 'c', 4: 'd', 5: 'e'}
```

Подсчет количества элементов

Пусть дан массив ['a', 'b', 'c', 'a', 'a', 'b']. Давайте подсчитаем количество одинаковых элементов в этом массиве и сделаем результат в виде объекта {a: 3, b: 2, c: 1}.

Для решения сделаем объект count с начальным значением {a: 0, b: 0, c: 0}. Будем перебирать массив циклом и увеличивать соответствующее значение в объекте count. К примеру: если текущий элемент массива - это 'a', то увеличим count['a'] на единицу - вот так: count['a']++.

Только вместо 'a' следует подставлять текущий элемент массива, вот так: count[arr[i]]++. Давайте напишем окончательный код:

```
var arr = ['a', 'b', 'c', 'a', 'a', 'b'];
var count = {a: 0, b: 0, c: 0};

for (var i = 0; i < arr.length; i++) {
    count[arr[i]]++;
}
```

```
console.log(count); //выведет {a: 3, b: 2, c: 1}
```

Пойдем дальше: то, что объекта **count** имеет изначальное значение {a: 0, b: 0, c: 0} - не очень удобно, ведь мы обычно не знаем, какие элементы есть в массиве.

Пусть объект **count** формируется автоматически таким образом: если такого элемента в count нет, то ему следует присвоить значение 1, а если есть - просто увеличить значение на единицу:

```
var arr = ['a', 'b', 'c', 'a', 'a', 'b'];
```

```
var count = {};
```

```
for (var i = 0; i < arr.length; i++) {  
    if (count[arr[i]] === undefined) {  
        count[arr[i]] = 1;  
    } else {  
        count[arr[i]]++;  
    }  
}
```

```
console.log(count); //выведет {a: 3, b: 2, c: 1}
```

Перебор многомерных массивов

Пусть дан следующий двухмерный массив:

```
var arr = [[1, 2, 3, 4, 5], [6, 7, 8], [9, 10]];
```

Давайте выведем все его элементы на экран. Для этого нам необходимо запустить два вложенных друг в друга цикла:

```
var arr = [[1, 2, 3, 4, 5], [6, 7, 8], [9, 10]];
```

```
for (var i = 0; i < arr.length; i++) {  
    for (var j = 0; j < arr[i].length; j++) {  
        alert(arr[i][j]);  
    }  
}
```

Задачи для решения

- 1 Заполните массив следующим образом: в первый элемент запишите 'x', во второй 'xx', в третий 'xxx' и так далее.
- 2 Заполните массив следующим образом: в первый элемент запишите '1', во второй '22', в третий '333' и так далее.

7 Сделайте функцию **arrayFill**, которая будет заполнять массив заданными значениями. Первым параметром функция принимает значение, которым заполнять массив, а вторым - сколько элементов должно быть в массиве. Пример: `arrayFill('x', 5)` сделает массив `['x', 'x', 'x', 'x', 'x']`.

8 Дан массив с числами. Узнайте сколько элементов с начала массива надо сложить, чтобы в сумме получилось больше 10-ти.

9 Дан массив с числами. Не используя метода `reverse` переверните его элементы в обратном порядке.